

OverView

- Automotive ECUs & Basics

- Vehicle Electrical Architecture

- Evolution of Vehicle Electrical Architecture

- Distributed Architecture

- Domain Architecture

- Zonal Architecture

- The QNX OS MicroKernel

- IPC

- Interrupt Redirector

- Scheduler

- Process, Threads and Synchronization

- Process Creation (Fork, Exec and Spawn)
 - Threads (pthread lib)
 - Scheduling Algorithm (FIFO, round-robin)
 - Cluster based scheduling
 - Priority Inheritance and Priority inversion handling
 - Synchronization (Mutex, Semaphore, Condvar, barriers)

OverView

- Inter-process communication

- Message Passing (Channel and Connection)

- Pulses

- Signals

- Events

- Shared Memory

- QNX Neutrino Boot Sequence

- Hardware Reset

- IPL

- Startup Code

- Procnto

- Boot Script

- Process Manager

- Process Management

- Memory Management

- Pathname Management

- Resource Manager

- Architecture & Communication

OverView

- QNX Hypervisor
 - Hypervisor Architecture
 - Host Domain VS guest VMS
- Device
 - Physics Device - Pass-through
 - Virtual Devices (Vdevs) - Para-virtualized
- Protection Feature
 - Isolation between host and guests (Temporal and Spatial)
- Configuration
 - Configuration the hypervisor host domain
 - Configuration qvm processes and VMs(guests)
- Communication
 - Networking
 - Shared Memory

Now Let's Deep Dive into VirtIO
Directly VirtIO? Let's start a Journey till VirtIO
from Virtualisation

Virtualization - One Machine, Many Worlds

One chip, many identities — Multiple independent operating systems run on the same physical hardware, each thinking it owns the machine.

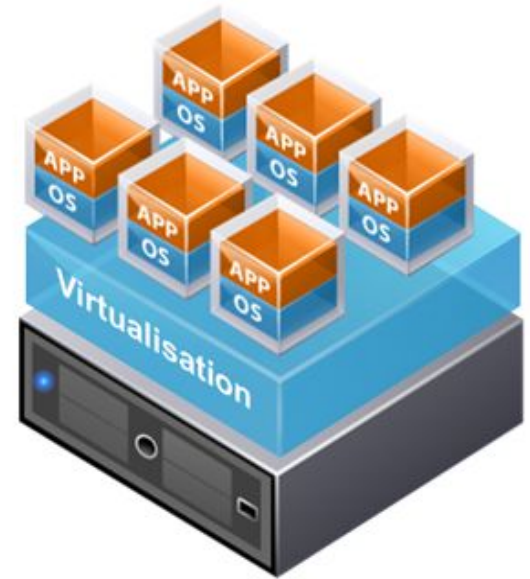
Software decides what hardware exists — CPU, memory, and devices are abstracted, so a "guest" gets what it needs without touching real silicon directly.

Isolation without duplication — No need for separate physical machines to keep workloads independent — separation happens in software, not in wiring.

Consolidation, not compromise — Fewer physical ECUs/chips, same (or better) reliability and separation guarantees.



Traditional Architecture



Virtual Architecture

Hypervisor - The Traffic Controller for Hardware

Sits between hardware and every OS —

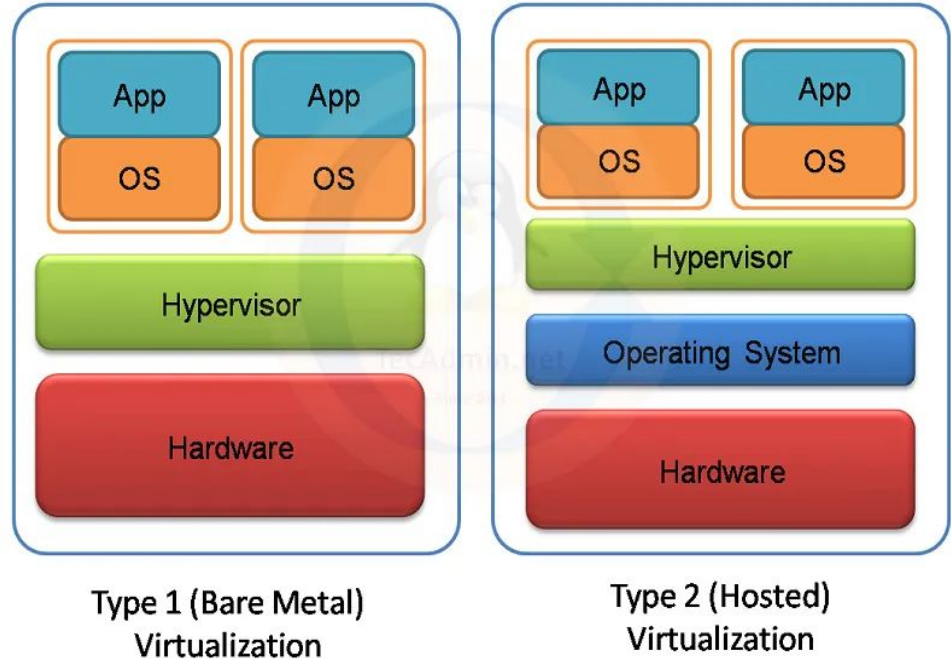
The hypervisor runs beneath all guest operating systems, controlling who gets access to what, and when.

Type 1 vs Type 2 —

Bare-metal hypervisors (Type 1) run directly on hardware for max performance and reliability; hosted hypervisors (Type 2) run on top of a host OS — automotive and safety-critical systems almost always use Type 1.

Enforces the walls, not just the sharing

— It's not only about splitting resources — it guarantees one guest crashing or misbehaving can never touch another.



QNX Hypervisor - Real-Time Meets Isolation

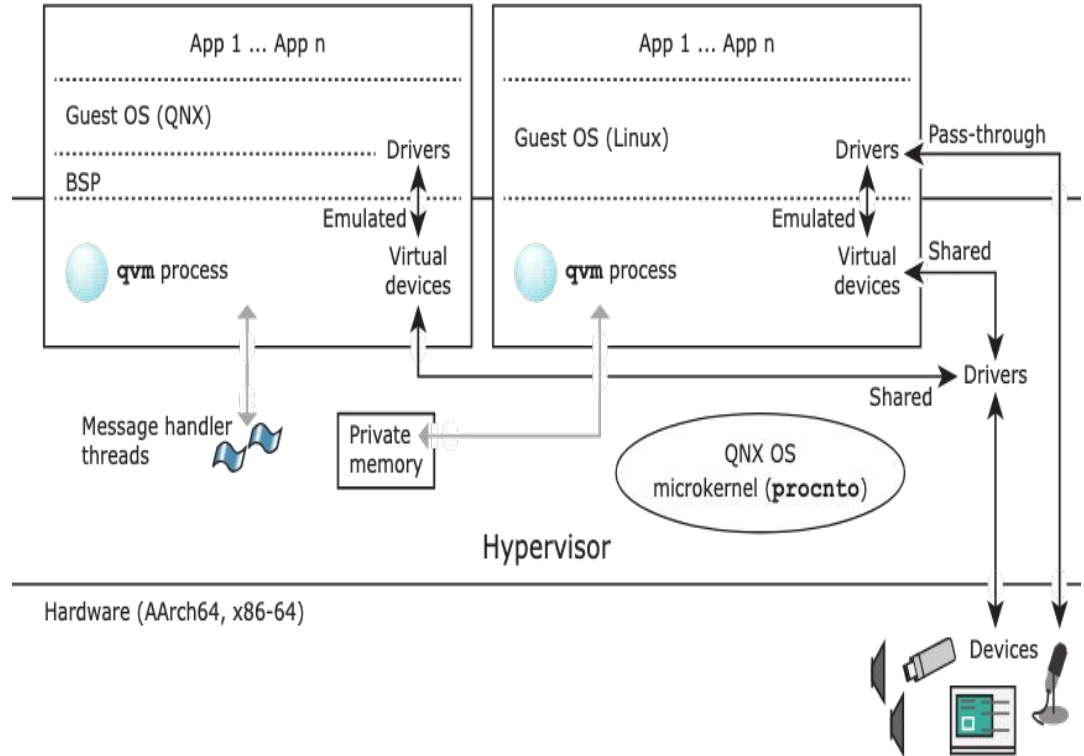
Built on the QNX microkernel itself —

Instead of a separate hypervisor codebase, QNX extended its already-trusted, safety-certified microkernel to also act as the hypervisor — same small, reliable core.

Host domain + guest VMs, side by side

— A privileged QNX host domain manages the system, while guest VMs (QNX, Linux, Android, etc.) run fully isolated on the same chip.

Two ways to reach a device — Physical pass-through for near-native performance on dedicated hardware, or virtual devices (VDevs) when a resource needs to be shared across guests.

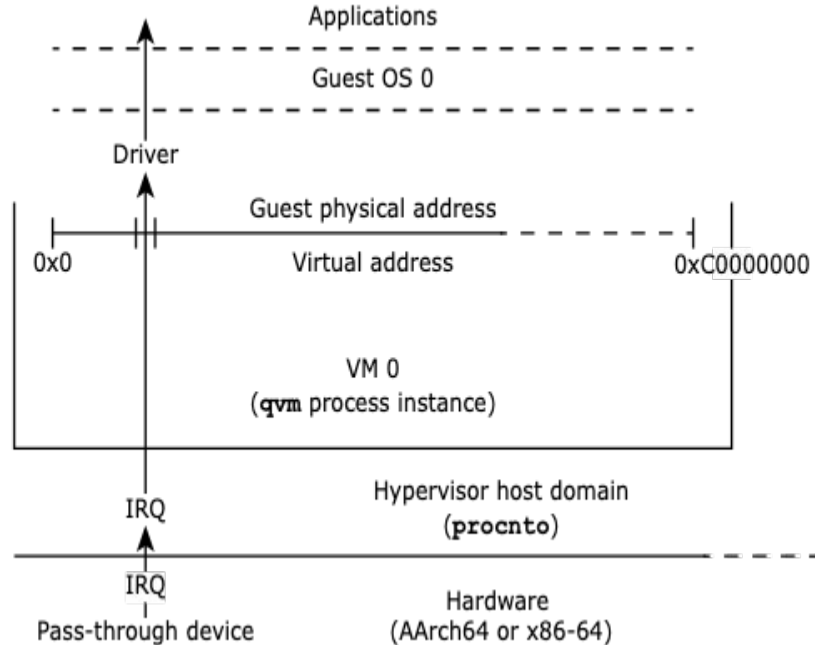


Physical Pass-Through — Direct Hardware Access

The device is handed entirely to one guest — The hypervisor assigns a real physical device directly to a specific guest VM, with no software emulation in between.

Near-native performance — Since the guest talks almost straight to the hardware, latency and overhead are minimal — critical for things like sensors, displays, or safety-relevant I/O.

No sharing — That device now belongs exclusively to one guest; no other guest can access it, even if it's idle.



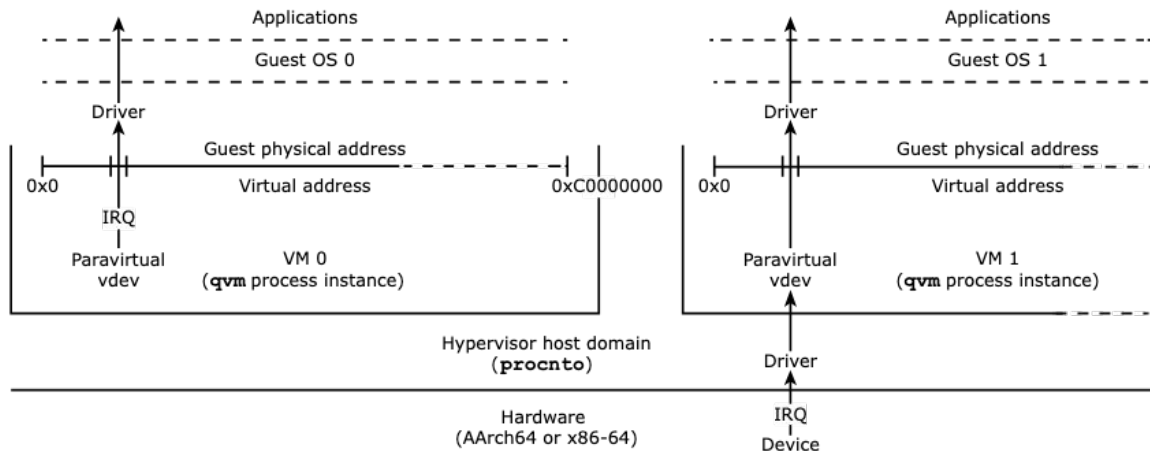
Para-Virtualization — The Guest Cooperates

The guest knows it's virtualized — Unlike full virtualization (where the hypervisor fakes real hardware perfectly), the guest OS is modified to be aware it's running on a hypervisor and works *with* it instead of being fooled.

Guest talks through a defined interface, not real hardware — Instead of the hypervisor emulating every register and behavior of a real device, the guest uses a lightweight, purpose-built interface to request I/O.

Much lower overhead than full emulation — Because there's no need to trap and translate every hardware-level instruction, para-virtualized I/O is significantly faster than full device emulation.

Requires guest OS support — This only works if the guest's drivers are written to speak this cooperative interface — which is exactly what VirtIO provides as a standard.

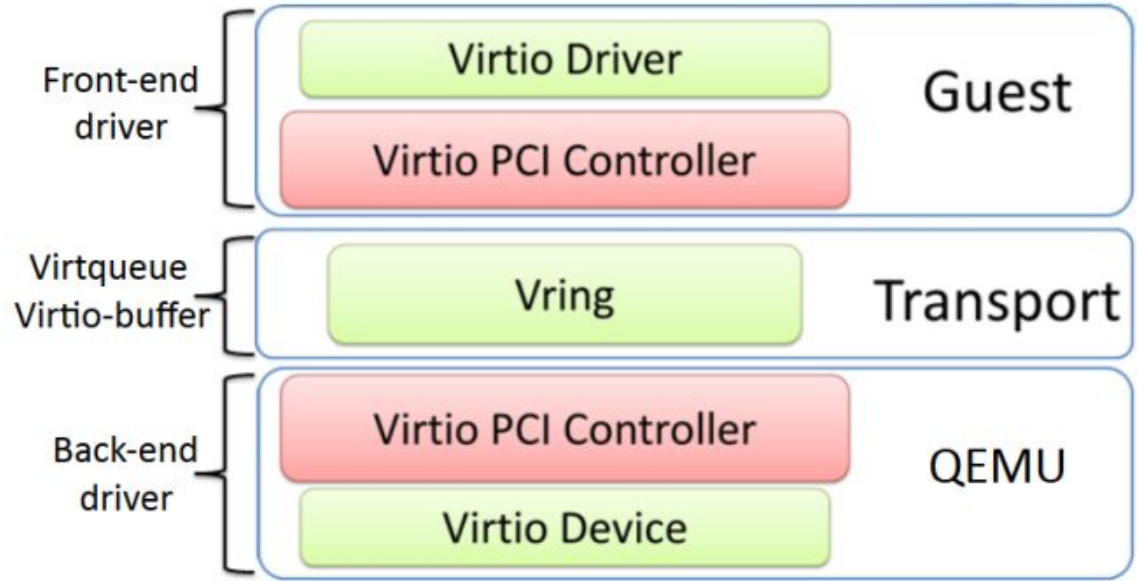


VirtIO — The Standard Language for Virtual Devices

A common interface for para-virtualized devices — Instead of every hypervisor and guest OS inventing its own way to talk to virtual devices, VirtIO defines one standard protocol that both sides agree to speak.

Built around virtqueues — Data moves between guest and hypervisor through shared, lock-free queues, letting the guest submit requests and the hypervisor process them efficiently, without heavy overhead.

One standard, many device types — The same core mechanism supports different device categories — storage, networking, console, and more — each built as a thin, device-specific layer on top of the same queue model.



virtio-blk flow — core steps

1. **Init device** — reset, ACKNOWLEDGE, DRIVER, negotiate features, DRIVER_OK
2. **Create virtqueue** — descriptor table + avail ring + used ring, register with device
3. **Build request** — 3 chained descriptors: header → data → status
4. **Publish + notify** — put descriptor index in avail ring, ring the doorbell
5. **Device does I/O** — reads/writes disk, fills data + status, updates used ring
6. **Interrupt + handle** — guest gets interrupt, reads used ring, processes result

virtio-console flow

1. **Init device** — reset, ACKNOWLEDGE, DRIVER, negotiate features (e.g. multiport), DRIVER_OK
2. **Create virtqueues** — 2 queues per port: rx (host→guest) and tx (guest→host)
3. **Build request** — tx: fill buffer with outgoing bytes (read-only descriptor). rx: pre-allocate empty buffers (write-only descriptors)
4. **Publish + notify** — tx: push per message. rx: push a pool of empty buffers ahead of time, notify device
5. **Device does I/O** — tx: device forwards bytes to host console. rx: device writes incoming bytes into a queued empty buffer
6. **Interrupt + handle** — tx: free descriptor. rx: read data out, then **re-queue buffer as empty again** (recycle, not free)